

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree	Branch	B.Tech Artificial Intelligence & Data Science
Semester	Subject Code	CS3807 – Deep Learning Laboratory
Academic Year		2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is:

$$\frac{N - F + 2P}{S} + 1$$

where

- N : Input Size
- F : Filter Size
- P : Padding
- S : Stride

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

Common Convolution Kernels

1. Identity Kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose: Preserves the original image and is used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications: Object detection, lane detection, and face recognition.

3. Sobel-Y Kernel (Horizontal Edge Detection)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Text detection, boundary extraction, and medical image analysis.

4. Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Edge enhancement, satellite image analysis, and medical imaging.

5. Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Image enhancement, Optical Character Recognition (OCR), and face recognition.

6. Box Blur Kernel

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications: Noise removal, image smoothing, and image preprocessing.

7. Gaussian Blur Kernel

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Noise reduction, computer vision preprocessing, and feature extraction.

8. Emboss Kernel

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications: Texture analysis and artistic image effects.

9. Outline Detection Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications: Image segmentation, boundary detection, and shape extraction.

10. Motion Blur Kernel

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications: Motion simulation, image restoration, and video processing.

Numerical Example 1: Convolution

Consider the following input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The resulting feature map is:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter, the output is:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

Numerical Example 3: Parameter Calculation

Suppose an input image has dimensions:

$$32 \times 32 \times 3$$

and the convolution layer contains:

- Filters = 16
- Kernel = 3×3

The number of parameters is:

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16 = 448$$

Therefore,

Total trainable parameters = 448

4. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

CIFAR-10 contains ten object categories:

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

Each image is a colour image of size 32×32 pixels with three RGB channels.

5. Image Preprocessing

The CIFAR-10 images are colour images represented using three RGB channels. The pixel values were normalized before training so that the input values were scaled to a suitable range for neural network training.

The preprocessing steps performed were:

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Inspect the dimensions of the training and testing sets.
3. Display sample images from the dataset.
4. Plot the distribution of the ten classes.
5. Normalize the image pixel values.
6. Preserve the image dimensions as $32 \times 32 \times 3$ for convolutional processing.

6. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Actual Implementation:

The CIFAR-10 dataset was loaded and its dimensions were verified. Ten sample images were displayed with their corresponding class labels. The class distribution was also plotted to verify the number of samples available for each class.

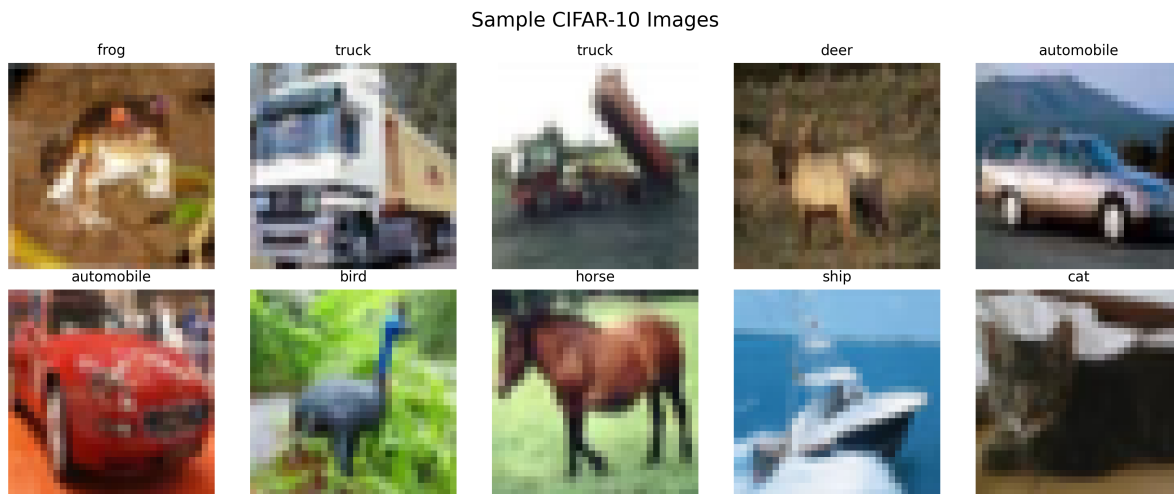


Figure 1: Sample CIFAR-10 images with their corresponding class labels.

Inference:

The sample images demonstrate the diversity of objects present in the CIFAR-10 dataset. The images are low-resolution RGB images of size $32 \times 32 \times 3$, making feature extraction important for distinguishing visually similar object categories.

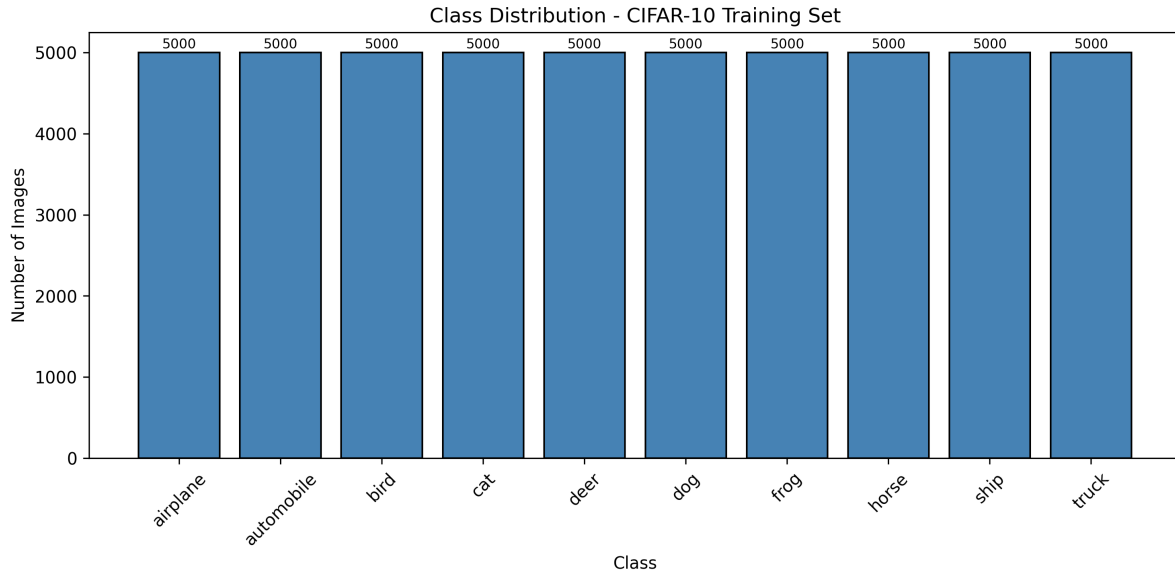


Figure 2: Class distribution of the CIFAR-10 training set.

Inference:

The class distribution is perfectly balanced, with each of the ten classes containing 5,000 training images. Therefore, there is no class imbalance in the training dataset and no additional class-weighting or resampling is required.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Actual Implementation:

A convolution operation was performed using three different kernel sizes. The resulting feature map dimensions were recorded as follows:

Kernel Size	Output Feature Map Size
3×3	30×30
5×5	28×28
7×7	26×26

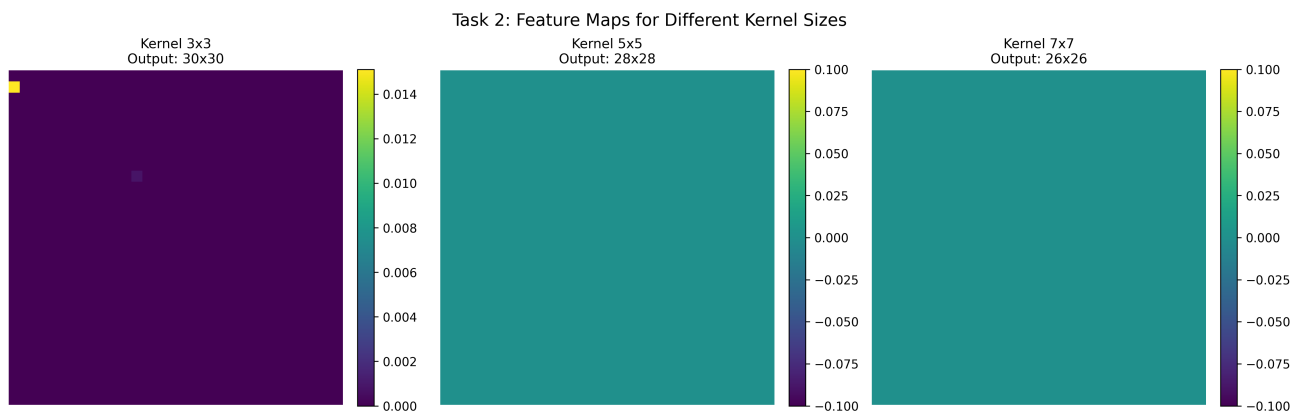


Figure 3: Comparison of feature map sizes for different convolution kernel sizes.

Inference:

Increasing the kernel size reduces the spatial dimensions of the feature map when valid padding is used. The 3×3 kernel produces the largest output of 30×30 , while the 7×7 kernel produces the smallest output of 26×26 .

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Actual Implementation:

The effect of stride and padding was studied using the following configurations:

Stride	Padding	Output Size
1	Valid	30×30
2	Valid	15×15
1	Same	32×32
2	Same	16×16

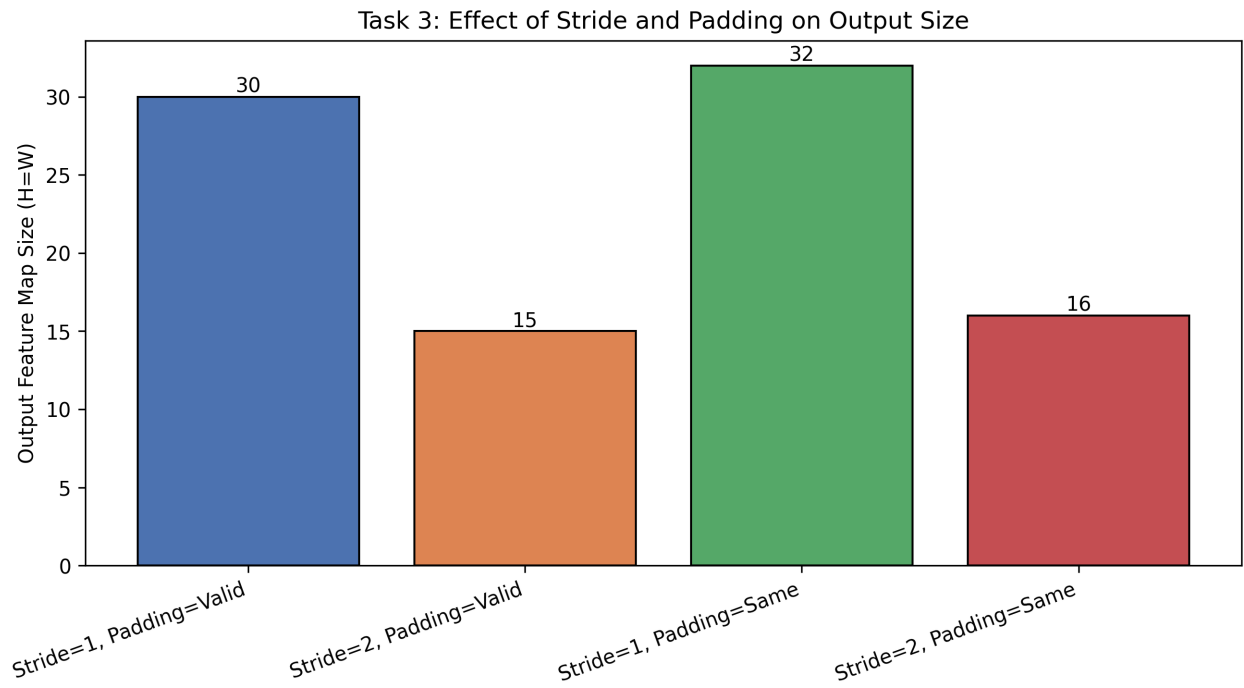


Figure 4: Effect of stride and padding on convolution output size.

Inference:

Increasing the stride from 1 to 2 reduces the spatial dimensions of the output feature map. Same padding preserves the spatial dimensions for stride 1, whereas valid padding reduces the dimensions because no padding is added around the input.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Actual Implementation:

A convolutional layer with 16 filters was applied to the input image, and eight of the resulting feature maps were visualized.

Task 4: Feature Maps After First Convolutional Layer

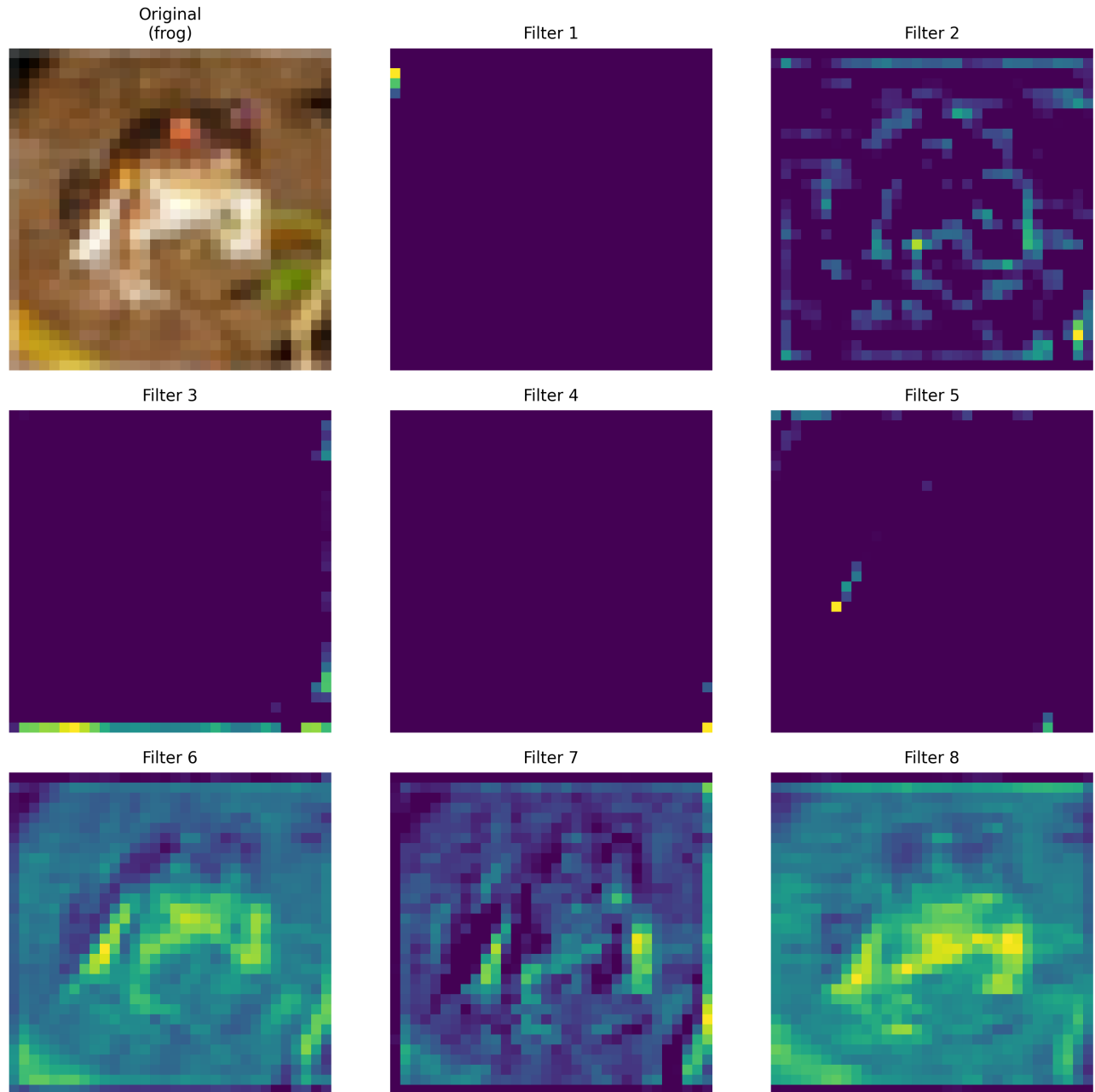


Figure 5: Feature maps obtained after the first convolutional layer.

Inference:

The feature maps show that different filters respond to different visual characteristics of the input image. Some filters produce localized activations, while others highlight broader structures and patterns, demonstrating how convolutional layers begin extracting useful visual features.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Actual Implementation:

Both pooling methods were applied using a 2×2 pooling window. The original feature map had dimensions 32×32 , while both pooling operations produced feature maps of dimensions 16×16 .

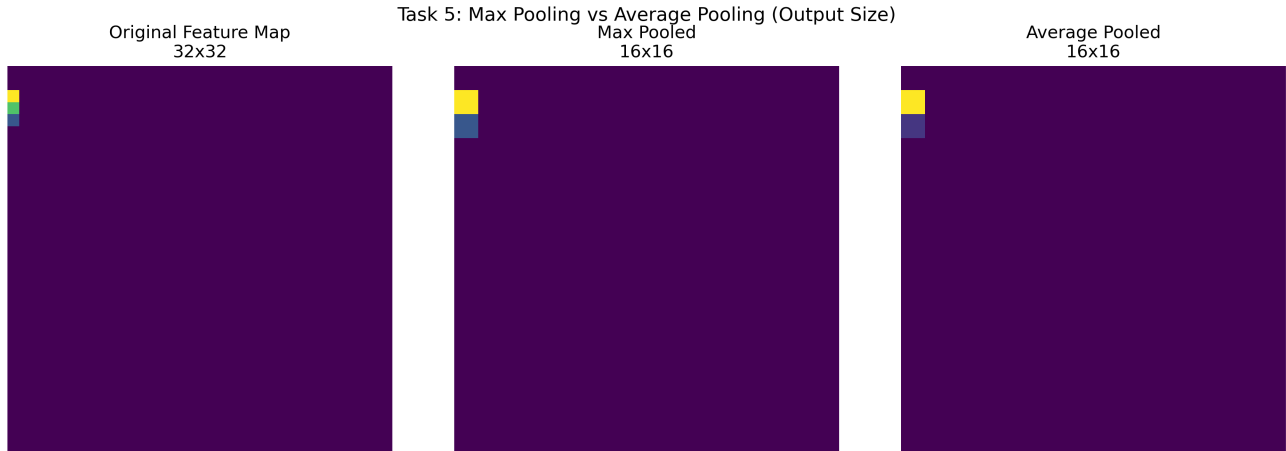


Figure 6: Comparison of original, max-pooled, and average-pooled feature maps.

Inference:

Both Max Pooling and Average Pooling reduce the spatial dimensions from 32×32 to 16×16 . Max Pooling retains the strongest activation within each pooling window, whereas Average Pooling computes the mean activation.

Pooling Accuracy Comparison

The classification accuracies obtained using the two pooling strategies were:

Pooling Method	Test Accuracy
Max Pooling	63.09%
Average Pooling	61.11%

Inference:

Max Pooling achieved a higher test accuracy of 63.09% compared with 61.11% for Average Pooling. This indicates that retaining the strongest activations was more effective for classification in this experiment.

Task 6

Construct the following CNN.

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Actual Implementation:

The CNN was implemented using TensorFlow/Keras with the following architecture:

Layer	Output / Configuration
Input	$32 \times 32 \times 3$
Conv2D	32 filters, 3×3 , ReLU, Same padding
MaxPooling2D	2×2
Conv2D	64 filters, 3×3 , ReLU, Same padding
MaxPooling2D	2×2
Flatten	4096 units
Dense	128 units, ReLU
Output Dense	10 units, Softmax

The model was trained using the Adam optimizer for 20 epochs with a batch size of 32. The total number of trainable parameters was:

$$545,098$$

The parameter calculation is:

$$(3 \times 3 \times 3 + 1) \times 32 = 896$$

$$(3 \times 3 \times 32 + 1) \times 64 = 18,496$$

After two 2×2 pooling layers:

$$32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$$

Therefore, the flattened representation is:

$$8 \times 8 \times 64 = 4096$$

The dense layer contains:

$$4096 \times 128 + 128 = 524,416$$

The output layer contains:

$$128 \times 10 + 10 = 1,290$$

Hence,

$$545,098$$

Training Accuracy

Inference:

The training accuracy rises smoothly throughout training, starting around 50% in epoch 1 and reaching approximately 97% by epoch 20. The curve climbs quickly in the first few epochs and then flattens out gradually, showing the model is fitting the training data more and more closely as training progresses, with almost no noise in the trend since it is computed over the full training set each epoch.

Validation Accuracy

Inference:

The validation accuracy increases quickly for the first 5 epochs, reaching roughly 70–72%, but then plateaus and fluctuates in a narrow band between about 70% and 72% for the remainder of training. Unlike the training accuracy, it does not continue to improve, and shows a mild downward drift after epoch 14–15, indicating that the model stops learning generalizable features fairly early even though training accuracy keeps rising.

Training Loss

Inference:

The training loss decreases steadily and smoothly from about 1.39 at epoch 1 to below 0.1 by epoch 20, with the steepest drop occurring in the first 7–8 epochs. The consistently decreasing curve confirms that the optimizer is successfully minimizing error on the training set throughout all 20 epochs.

Validation Loss

Inference:

The validation loss initially decreases along with the training loss, reaching its lowest point (about 0.86) around epoch 4–5. After this point it rises almost continuously, reaching about 2.19 by epoch 20, even while validation accuracy stays roughly flat. This divergence between a falling training loss and a rising validation loss is a classic sign of overfitting: the model becomes increasingly overconfident on the training data while its predictions on unseen data grow less reliable.

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Actual Implementation:

The trained CNN was evaluated on the CIFAR-10 test set using accuracy, precision, recall, F1-score, confusion matrix, and classification report.

The obtained overall performance was:

Metric	Result
Test Accuracy	69.40%
Macro Precision	69.97%
Macro Recall	69.40%
Macro F1-score	69.23%
Test Loss	2.1924
Trainable Parameters	545,098

Confusion Matrix

Inference:

The confusion matrix shows that most misclassifications occur between visually and semantically similar animal classes. The largest single confusion is cats being predicted as dogs (304 instances), while

dogs are also confused with cats (95 instances) to a lesser extent. Bird is frequently confused with cat, deer, and dog, and deer is often mixed up with horse. Vehicle classes (airplane, automobile, ship, truck) are classified far more accurately, with automobile and truck showing some mutual confusion, which makes sense given their similar shapes and backgrounds compared to animal classes.

Per-Class Accuracy

Inference:

Frog (0.80), truck (0.79), and automobile/horse (0.78) achieve the highest per-class accuracy, while cat (0.41) and bird (0.53) are the weakest-performing classes by a clear margin. This matches the confusion matrix findings: cat has the most misclassifications, primarily being confused with dog, dragging its accuracy well below the overall average of 69.4%, while the more visually distinct vehicle and frog classes remain consistently well classified.

7. Mandatory Plots

The following plots were generated as part of the experiment:

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Pooling Comparison
9. Confusion Matrix
10. Per-Class Accuracy

The corresponding figures and inferences are included in the experimental procedure and results sections.

8. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Using:

$$\frac{N - F + 2P}{S} + 1$$

where:

$$N = 64, \quad F = 5, \quad P = 2, \quad S = 2$$

Therefore,

$$\frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1$$

Using the floor operation:

$$\boxed{\text{Output Size} = 32}$$

Hence, the output feature map has spatial dimensions:

$$32 \times 32$$

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

For an RGB input, the number of input channels is 3.

$$(3 \times 3 \times 3 + 1) \times 64$$

$$= (27 + 1) \times 64$$

$$1792$$

Therefore, the convolution layer contains 1,792 trainable parameters.

3. Compare ReLU and Sigmoid activation functions.

A comparison between ReLU and Sigmoid activation functions was performed experimentally.

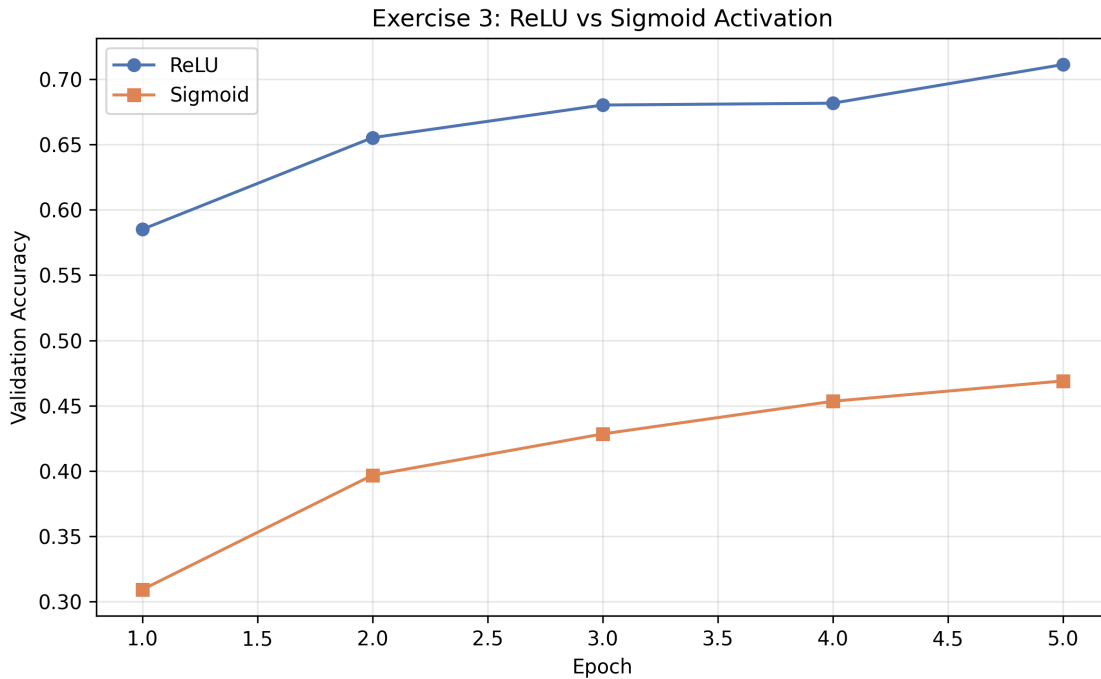


Figure 7: Validation accuracy comparison between ReLU and Sigmoid activations over 5 epochs.

Inference:

The ReLU-activated network reaches roughly 71% validation accuracy by epoch 5, while the Sigmoid-activated network only reaches about 47% in the same number of epochs, starting from a much lower point (around 31%) in epoch 1. ReLU converges noticeably faster and to a much higher accuracy because it avoids the vanishing-gradient problem that Sigmoid suffers from in deeper networks, where gradients shrink as they backpropagate through saturated Sigmoid activations, slowing down learning considerably.

4. Replace Max Pooling with Average Pooling and compare the results.

Average Pooling was implemented and compared with Max Pooling in terms of output size and classification accuracy.

The output spatial dimensions for both methods were:

$$32 \times 32 \rightarrow 16 \times 16$$

The measured accuracy values were:

Pooling Method	Output Size	Test Accuracy
Max Pooling	16×16	63.09%
Average Pooling	16×16	61.11%

Inference:

In the pooling comparison experiment, both pooling methods reduced the feature-map dimensions from 32×32 to 16×16 . Max Pooling achieved higher test accuracy than Average Pooling in this experiment.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

A comparison was performed by increasing the number of convolution filters from 16 to 64.

The recorded results were:

Number of Filters	Parameters	Validation Accuracy	Training Time
16	268,650	68.00%	21.6 s
64	1,125,642	70.72%	28.7 s

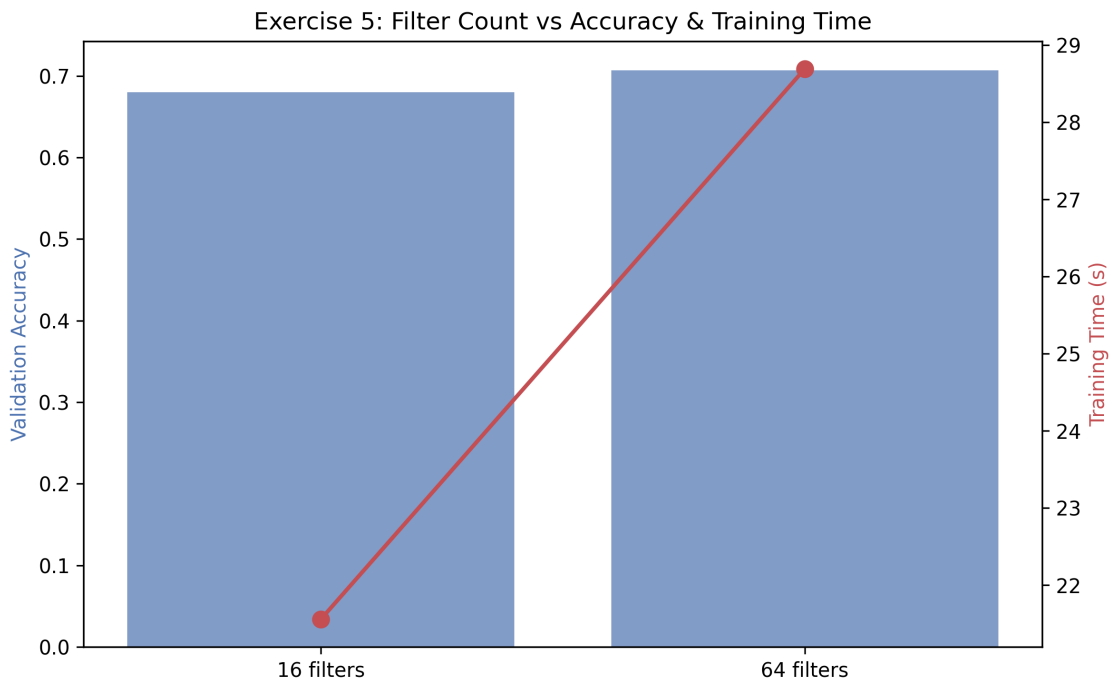


Figure 8: Effect of increasing convolution filters from 16 to 64 on validation accuracy and training time.

Inference:

Increasing the number of filters from 16 to 64 improved validation accuracy from 68.00% to 70.72%, but at the cost of roughly 4.2 times more trainable parameters and about 33% longer training time (21.6s to 28.7s for 5 epochs). This illustrates the typical accuracy–computation trade-off in CNN design: more filters allow the network to learn a richer set of features and improve performance, but with diminishing returns relative to the increase in parameters and training cost, so the optimal filter count depends on the available compute budget and how much accuracy improvement is actually needed.

9. Results

The CNN was successfully implemented and trained on the CIFAR-10 dataset.

Training Accuracy	97.20% (final epoch)
Testing Accuracy	69.40%
Precision (Macro)	69.97%
Recall (Macro)	69.40%
F1-score (Macro)	69.23%
Number of Parameters	545,098
Test Loss	2.1924

The CNN architecture consisted of two convolutional layers followed by ReLU activation and max pooling, a flattening layer, a fully connected layer, and a softmax output layer. The model was trained using the Adam optimizer for 20 epochs with a batch size of 32.

The lowest validation loss observed during training was 0.8616 at epoch 4. The subsequent increase in validation loss indicates the presence of overfitting during later epochs.

10. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred for image data because it preserves the spatial structure of the image and learns local patterns using shared filters. The same filter can detect a particular feature at different spatial locations, resulting in fewer parameters compared with directly connecting every input pixel to every neuron in a fully connected layer.

2. How does stride affect the feature map size?

Stride determines the number of pixels by which the convolution kernel moves across the input. Increasing the stride reduces the spatial dimensions of the output feature map. For example, the experiment produced smaller feature maps when the stride was increased from 1 to 2.

3. What is the role of padding?

Padding adds additional pixels around the boundary of the input before convolution. Same padding helps preserve spatial dimensions, while valid padding does not add padding and therefore produces a smaller feature map.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps, thereby reducing computational cost and the number of subsequent parameters. It also helps retain important information while making the learned representation more compact.

5. How do feature maps represent image characteristics?

Feature maps represent the response of learned convolution filters to different regions of an image. Different filters can respond to edges, textures, shapes, and other visual patterns. The feature-map visualizations demonstrate that different filters produce different activation patterns for the same input image.

6. Why do CNNs require fewer parameters than MLPs?

CNNs use local connectivity and parameter sharing. A convolution filter is applied across different spatial locations using the same set of weights. In contrast, an MLP requires separate weights for connections between every input feature and neuron, resulting in substantially more parameters for image data.

11. Conclusion

This experiment implemented a Convolutional Neural Network for multi-class image classification using the CIFAR-10 dataset and TensorFlow/Keras. The experiment demonstrated the effect of kernel size, stride, padding, pooling strategy, and convolutional filters on feature-map dimensions and model behaviour.

Feature maps generated after convolution showed how different filters learn different visual patterns from an input image. Max Pooling and Average Pooling were compared, and both were observed to reduce spatial dimensions while retaining important information.

The final CNN consisted of two convolutional layers, ReLU activation, max pooling, a fully connected layer, and a softmax output layer. The model was trained using Adam for 20 epochs with a batch size of 32 and contained 545,098 trainable parameters. The model achieved a test accuracy of 69.40%, with macro precision of 69.97%, macro recall of 69.40%, and macro F1-score of 69.23%.

The comparison of pooling methods showed that Max Pooling achieved higher test accuracy than Average Pooling. Increasing the number of convolution filters increased the validation accuracy from 68.00% to 70.72%, but also increased the number of parameters and training time.

Overall, the experiment provided practical understanding of CNN architecture design, convolution-based feature extraction, pooling, hyperparameter effects, parameter calculation, and classification performance evaluation.

12. References

1. TensorFlow Documentation.
2. CIFAR-10 Dataset Documentation.

GitHub Repository

<https://github.com/D-DRM/Deep-Learning-Lab>